

Data Structures Lab: Doubly & Circular Linked Lists

Here are **4 practical lab questions with complete C programs**, covering the operations and complexity topics from the syllabus.

Lab 1: Doubly Linked List — All Basic Operations

Question

Write a menu-driven C program to implement a Doubly Linked List and perform the following operations:

1. Insert at beginning
2. Insert at end
3. Insert after a given element
4. Delete from beginning
5. Delete from end
6. Delete a given element
7. Display in forward direction
8. Display in reverse direction
9. Search an element
10. Count the number of nodes

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *head = NULL;
struct Node *tail = NULL;

/* Insert at beginning */
void insertBeginning(int value)
{
    struct Node *newNode;

    newNode = malloc(sizeof(struct Node));
```

```

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;

    if (head == NULL)
    {
        head = tail = newNode;
    }
    else
    {
        head->prev = newNode;
        head = newNode;
    }

    printf("%d inserted at beginning.\n", value);
}

/* Insert at end */
void insertEnd(int value)
{
    struct Node *newNode;

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = tail;

    if (tail == NULL)
    {
        head = tail = newNode;
    }
    else
    {
        tail->next = newNode;
        tail = newNode;
    }

    printf("%d inserted at end.\n", value);
}

```

```

}

/* Insert after a given element */
void insertAfter(int key, int value)
{
    struct Node *temp = head;
    struct Node *newNode;

    while (temp != NULL && temp->data != key)
        temp = temp->next;

    if (temp == NULL)
    {
        printf("%d not found.\n", key);
        return;
    }

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->prev = temp;
    newNode->next = temp->next;

    if (temp->next != NULL)
        temp->next->prev = newNode;
    else
        tail = newNode;

    temp->next = newNode;

    printf("%d inserted after %d.\n", value, key);
}

/* Forward traversal */
void displayForward()
{
    struct Node *temp = head;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    printf("Forward: ");

```

```

while (temp != NULL)
{
    printf("%d <-> ", temp->data);
    temp = temp->next;
}

printf("NULL\n");
}

/* Reverse traversal */
void displayReverse()
{
    struct Node *temp = tail;

    if (tail == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    printf("Reverse: ");

    while (temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }

    printf("NULL\n");
}

/* Search */
void search(int value)
{
    struct Node *temp = head;
    int position = 1;

    while (temp != NULL)
    {
        if (temp->data == value)
        {
            printf("%d found at position %d.\n",
                value, position);
            return;
        }

        temp = temp->next;
        position++;
    }

    printf("%d not found.\n", value);
}

```

```

/* Delete from beginning */
void deleteBeginning()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    head = head->next;

    if (head == NULL)
        tail = NULL;
    else
        head->prev = NULL;

    printf("%d deleted.\n", temp->data);

    free(temp);
}

/* Delete from end */
void deleteEnd()
{
    struct Node *temp;

    if (tail == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temp = tail;
    tail = tail->prev;

    if (tail == NULL)
        head = NULL;
    else
        tail->next = NULL;

    printf("%d deleted.\n", temp->data);

    free(temp);
}

/* Delete a given element */
void deleteValue(int value)
{

```

```

    struct Node *temp = head;

    while (temp != NULL && temp->data != value)
        temp = temp->next;

    if (temp == NULL)
    {
        printf("%d not found.\n", value);
        return;
    }

    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else
        head = temp->next;

    if (temp->next != NULL)
        temp->next->prev = temp->prev;
    else
        tail = temp->prev;

    free(temp);

    printf("%d deleted.\n", value);
}

/* Count nodes */
void countNodes()
{
    struct Node *temp = head;
    int count = 0;

    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }

    printf("Number of nodes = %d\n", count);
}

int main()
{
    int choice, value, key;

    while (1)
    {
        printf("\n===== DOUBLY LINKED LIST =====\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert After a Given Element\n");
        printf("4. Delete from Beginning\n");
    }
}

```

```
printf("5. Delete from End\n");
printf("6. Delete a Given Element\n");
printf("7. Display Forward\n");
printf("8. Display Reverse\n");
printf("9. Search\n");
printf("10. Count Nodes\n");
printf("11. Exit\n");

printf("Enter choice: ");
scanf("%d", &choice);

switch (choice)
{
case 1:
    printf("Enter value: ");
    scanf("%d", &value);
    insertBeginning(value);
    break;

case 2:
    printf("Enter value: ");
    scanf("%d", &value);
    insertEnd(value);
    break;

case 3:
    printf("Enter existing element: ");
    scanf("%d", &key);

    printf("Enter new value: ");
    scanf("%d", &value);

    insertAfter(key, value);
    break;

case 4:
    deleteBeginning();
    break;

case 5:
    deleteEnd();
    break;

case 6:
    printf("Enter value to delete: ");
    scanf("%d", &value);
    deleteValue(value);
    break;

case 7:
    displayForward();
    break;
```

```

        case 8:
            displayReverse();
            break;

        case 9:
            printf("Enter value to search: ");
            scanf("%d", &value);
            search(value);
            break;

        case 10:
            countNodes();
            break;

        case 11:
            exit(0);

        default:
            printf("Invalid choice.\n");
    }
}

```

Example

Forward: 10 <-> 20 <-> 30 <-> NULL

Reverse: 30 <-> 20 <-> 10 <-> NULL

Lab 2: Circular Singly Linked List

Question

Write a menu-driven C program to implement a Circular Singly Linked List with the following operations:

1. Insert at beginning
2. Insert at end
3. Insert after a given element
4. Delete from beginning
5. Delete from end
6. Delete a given element
7. Search
8. Display
9. Count nodes

Use both `HEAD` and `TAIL` pointers.

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *head = NULL;
struct Node *tail = NULL;

/* Insert at beginning */
void insertBeginning(int value)
{
    struct Node *newNode;

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;

    if (head == NULL)
    {
        head = tail = newNode;
        newNode->next = head;
    }
    else
    {
        newNode->next = head;
        head = newNode;
        tail->next = head;
    }

    printf("%d inserted at beginning.\n", value);
}

/* Insert at end */
void insertEnd(int value)
{

```

```

    struct Node *newNode;

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;

    if (head == NULL)
    {
        head = tail = newNode;
        newNode->next = head;
    }
    else
    {
        newNode->next = head;
        tail->next = newNode;
        tail = newNode;
    }

    printf("%d inserted at end.\n", value);
}

/* Insert after a given element */
void insertAfter(int key, int value)
{
    struct Node *temp;
    struct Node *newNode;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temp = head;

    do
    {
        if (temp->data == key)
            break;

        temp = temp->next;
    }
    while (temp != head);

    if (temp->data != key)
    {

```

```

        printf("%d not found.\n", key);
        return;
    }

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
        return;

    newNode->data = value;
    newNode->next = temp->next;
    temp->next = newNode;

    if (temp == tail)
        tail = newNode;

    printf("%d inserted after %d.\n", value, key);
}

/* Display */
void display()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temp = head;

    printf("Circular List: ");

    do
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    while (temp != head);

    printf("(HEAD)\n");
}

/* Search */
void search(int value)
{
    struct Node *temp;
    int position = 1;

    if (head == NULL)
    {

```

```

        printf("List is empty.\n");
        return;
    }

    temp = head;

    do
    {
        if (temp->data == value)
        {
            printf("%d found at position %d.\n",
                value, position);
            return;
        }

        temp = temp->next;
        position++;
    }
    while (temp != head);

    printf("%d not found.\n", value);
}

/* Delete from beginning */
void deleteBeginning()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    if (head == tail)
    {
        printf("%d deleted.\n", head->data);
        free(head);

        head = tail = NULL;
        return;
    }

    temp = head;
    head = head->next;
    tail->next = head;

    printf("%d deleted.\n", temp->data);

    free(temp);
}

```

```

/* Delete from end */
void deleteEnd()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    if (head == tail)
    {
        printf("%d deleted.\n", tail->data);
        free(tail);

        head = tail = NULL;
        return;
    }

    temp = head;

    while (temp->next != tail)
        temp = temp->next;

    printf("%d deleted.\n", tail->data);

    temp->next = head;
    free(tail);

    tail = temp;
}

/* Delete a given element */
void deleteValue(int value)
{
    struct Node *current;
    struct Node *previous;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    current = head;
    previous = tail;

    do
    {
        if (current->data == value)
            break;

```

```

        previous = current;
        current = current->next;
    }
    while (current != head);

    if (current->data != value)
    {
        printf("%d not found.\n", value);
        return;
    }

    if (current == head && current == tail)
    {
        head = tail = NULL;
    }
    else
    {
        previous->next = current->next;

        if (current == head)
            head = current->next;

        if (current == tail)
            tail = previous;
    }

    free(current);

    if (tail != NULL)
        tail->next = head;

    printf("%d deleted.\n", value);
}

/* Count nodes */
void countNodes()
{
    struct Node *temp;
    int count = 0;

    if (head == NULL)
    {
        printf("Number of nodes = 0\n");
        return;
    }

    temp = head;

    do
    {
        count++;
    }

```

```

        temp = temp->next;
    }
    while (temp != head);

    printf("Number of nodes = %d\n", count);
}

int main()
{
    int choice, value, key;

    while (1)
    {
        printf("\n===== CIRCULAR LINKED LIST =====\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert After a Given Element\n");
        printf("4. Delete from Beginning\n");
        printf("5. Delete from End\n");
        printf("6. Delete a Given Element\n");
        printf("7. Search\n");
        printf("8. Display\n");
        printf("9. Count Nodes\n");
        printf("10. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insertBeginning(value);
                break;

            case 2:
                printf("Enter value: ");
                scanf("%d", &value);
                insertEnd(value);
                break;

            case 3:
                printf("Enter existing element: ");
                scanf("%d", &key);

                printf("Enter new value: ");
                scanf("%d", &value);

                insertAfter(key, value);
                break;

```

```

        case 4:
            deleteBeginning();
            break;

        case 5:
            deleteEnd();
            break;

        case 6:
            printf("Enter value to delete: ");
            scanf("%d", &value);
            deleteValue(value);
            break;

        case 7:
            printf("Enter value to search: ");
            scanf("%d", &value);
            search(value);
            break;

        case 8:
            display();
            break;

        case 9:
            countNodes();
            break;

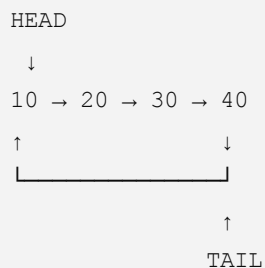
        case 10:
            exit(0);

        default:
            printf("Invalid choice.\n");
    }
}

```

Important point

For a circular linked list:



Always:

C

```
tail->next = head;
```

There is no `NULL` at the end.

Lab 3: Circular Doubly Linked List

Question

Write a C program to implement a Circular Doubly Linked List. Perform insertion at beginning and end, deletion from beginning and end, forward traversal and backward traversal.

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *head = NULL;
struct Node *tail = NULL;

/* Insert at beginning */
void insertBeginning(int value)
{
    struct Node *newNode;

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
        return;

    newNode->data = value;

    if (head == NULL)
    {
        head = tail = newNode;

        newNode->next = newNode;
```

```

        newNode->prev = newNode;
    }
    else
    {
        newNode->next = head;
        newNode->prev = tail;

        head->prev = newNode;
        tail->next = newNode;

        head = newNode;
    }
}

/* Insert at end */
void insertEnd(int value)
{
    struct Node *newNode;

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
        return;

    newNode->data = value;

    if (head == NULL)
    {
        head = tail = newNode;

        newNode->next = newNode;
        newNode->prev = newNode;
    }
    else
    {
        newNode->next = head;
        newNode->prev = tail;

        tail->next = newNode;
        head->prev = newNode;

        tail = newNode;
    }
}

/* Forward traversal */
void displayForward()
{
    struct Node *temp;

    if (head == NULL)
    {

```

```

        printf("List is empty.\n");
        return;
    }

    temp = head;

    printf("Forward: ");

    do
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    while (temp != head);

    printf(" (HEAD) \n");
}

/* Backward traversal */
void displayBackward()
{
    struct Node *temp;

    if (tail == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temp = tail;

    printf("Backward: ");

    do
    {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
    while (temp != tail);

    printf(" (TAIL) \n");
}

/* Delete from beginning */
void deleteBeginning()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

```

```

    }

    if (head == tail)
    {
        free(head);
        head = tail = NULL;
        return;
    }

    temp = head;

    head = head->next;

    head->prev = tail;
    tail->next = head;

    free(temp);
}

/* Delete from end */
void deleteEnd()
{
    struct Node *temp;

    if (tail == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    if (head == tail)
    {
        free(tail);
        head = tail = NULL;
        return;
    }

    temp = tail;

    tail = tail->prev;

    tail->next = head;
    head->prev = tail;

    free(temp);
}

int main()
{
    int choice, value;

    while (1)

```

```

{
    printf("\n===== CIRCULAR DOUBLY LINKED LIST =====\n");
    printf("1. Insert at Beginning\n");
    printf("2. Insert at End\n");
    printf("3. Delete from Beginning\n");
    printf("4. Delete from End\n");
    printf("5. Display Forward\n");
    printf("6. Display Backward\n");
    printf("7. Exit\n");

    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            printf("Enter value: ");
            scanf("%d", &value);
            insertBeginning(value);
            break;

        case 2:
            printf("Enter value: ");
            scanf("%d", &value);
            insertEnd(value);
            break;

        case 3:
            deleteBeginning();
            break;

        case 4:
            deleteEnd();
            break;

        case 5:
            displayForward();
            break;

        case 6:
            displayBackward();
            break;

        case 7:
            exit(0);

        default:
            printf("Invalid choice.\n");
    }
}

return 0;

```

Lab 4: Compare Doubly and Circular Linked Lists

Question

Write a C program that demonstrates the basic operations of both a Doubly Linked List and a Circular Singly Linked List. Perform insertion, deletion, searching and traversal and display the result of each operation.

Required operations

Doubly Linked List:

- Insert beginning
- Insert end
- Delete beginning
- Delete end
- Forward traversal
- Reverse traversal
- Search

Circular Linked List:

- Insert beginning
- Insert end
- Delete beginning
- Delete end
- Traversal
- Search

Analysis to be written in the lab record

Operation	Doubly LL	Circular SLL with Tail
Traversal	$O(n)$	$O(n)$
Searching	$O(n)$	$O(n)$
Insert beginning	$O(1)$	$O(1)$
Insert end	$O(1)$	$O(1)$
Delete beginning	$O(1)$	$O(1)$
Delete end	$O(1)$	$O(n)$
Reverse traversal	$O(n)$	Not directly possible

Operation	Doubly LL	Circular SLL with Tail
Extra pointer	prev + next	next

Recommended Lab Sequence

For your students, I would arrange the experiments in this order:

Experiment 1: Singly Linked List — Traversal, Searching, Insertion and Deletion

↓

Experiment 2: Stack using Linked List

↓

Experiment 3: Queue using Linked List

↓

Experiment 4: Doubly Linked List — Forward/Reverse Traversal and Operations

↓

Experiment 5: Circular Singly Linked List — All Operations

↓

Experiment 6: Circular Doubly Linked List — Operations and Traversal

This sequence moves from **basic linked-list concepts** → **applications** → **advanced linked-list structures**.